

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : CSA0921 – Programming in Java

Assignment Title:	Smart Campus Resource Allocation and Facility Management System using Java
Course Outcome(s) – CO:	CO1: Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4) CO2: Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4) CO3: Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques (BL3,BL4)
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure SDG 4 – Quality Education SDG 11 – Sustainable Cities and Communities

Problem Statement : A large autonomous university manages shared facilities such as computer laboratories, seminar halls, meeting rooms, multimedia studios, and innovation centers. Students, faculty, and researchers submit reservation requests for classes, workshops, examinations, and project activities. The current manual booking system often leads to scheduling conflicts, duplicate reservations, and inefficient utilization of campus resources. The University ICT Cell plans to develop a Java-based Smart Campus Resource Allocation and Facility Management System

to automate the reservation process. The application should maintain user profiles, manage facility details, process bookings, modify or cancel reservations, and generate utilization reports for administrators. The system must apply Object-Oriented Programming principles by modeling users and facilities through classes, inheritance, encapsulation, and polymorphism. Resource records and booking history should be managed using the Java Collection Framework with generics, lists, maps, and iterators. To support multiple users accessing the system simultaneously, the application should implement multithreading, synchronization, inter-thread communication, and appropriate exception handling. The final solution should be a menu-driven application that ensures efficient, reliable, and conflict-free campus resource management.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
1. Requirement Analysis, Java Fundamentals & Program Logic	15	Clearly understands all requirements, entities, workflows, and user interactions; uses appropriate data types, operators, conditional statements, loops, and decision-making structures efficiently and correctly.	Most requirements and workflows are identified; Java fundamentals and control structures are used correctly with minor errors.	Basic requirements are identified; fundamental constructs are used with some logical errors or inefficiencies.	Requirements are poorly understood; frequent errors in Java fundamentals and program logic affect correctness.
2. Class Design, Encapsulation, Inheritance & Polymorphism	20	Well-structured classes for users, facilities, bookings, and management; strong encapsulation and data hiding;	Classes are reasonably designed with appropriate encapsulation and inheritance; polymorphism	Basic classes and inheritance are present, but encapsulation or polymorphic behavior is incomplete.	Poor class organization; weak encapsulation; inheritance or polymorphism is absent or

		appropriate inheritance, method overriding, and runtime polymorphism are implemented effectively.	is implemented with minor limitations.		incorrectly implemented.
3. Java Collection Framework, Generics & Data Management	20	Appropriate use of List, ArrayList, Set, HashSet, Map, and HashMap with generics; iterators are effectively used for storing, searching, updating, and processing campus resources and booking records.	Appropriate collections and generics are used with minor implementation or retrieval issues.	Collections are used for basic storage and retrieval, but some data structures or iterator operations are inappropriate or incomplete.	Collections, generics, or iterators are largely missing, incorrectly implemented, or inefficiently used.
4. Multithreading, Synchronization, Exception Handling & Menu-Driven Functionality	25	Multiple threads effectively handle concurrent booking, cancellation, and facility monitoring; synchronization and inter-thread communication prevent race conditions; comprehensive exception handling manages invalid inputs, duplicate bookings, unavailable facilities, and scheduling conflicts; all menu operations function reliably.	Multithreading, synchronization, exception handling, and menu operations are mostly implemented correctly with minor concurrency or runtime issues.	Basic multithreading and exception handling are present, but synchronization, communication mechanisms, or some menu functions are incomplete.	Multithreading or synchronization is absent/non-functional; exception handling is inadequate and major menu operations are missing or unreliable.

5. System Integration, Reports, Documentation & Overall Quality	10	All modules are fully integrated; facility availability, booking statistics, utilization reports, and other required outputs are generated correctly; code is organized, readable, documented, and professionally presented.	Modules are well integrated and reports are mostly correct; code organization and documentation are satisfactory.	Basic integration and reporting are achieved, but some reports, documentation, or code quality need improvement.	Poor integration; reports are missing or incorrect; code is difficult to understand and documentation is inadequate.
6. Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Provides thoughtful justification of design decisions; clearly connects the system with relevant SDGs such as SDG 4, SDG 9, and SDG 11 ; provides specific reflection on challenges, problem-solving, and learning outcomes.	Provides reasonable justification of design choices and SDG relevance; discusses challenges and learning with adequate detail.	Reflection is present but generic; limited justification of design decisions, SDG relevance, or learning outcomes.	Reflection is missing or lacks meaningful discussion of design decisions, SDG relevance, challenges, or learning outcomes.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

1. Problem Statement

A large university manages shared facilities such as computer laboratories, seminar halls, meeting rooms, multimedia studios, and innovation centres. The existing manual booking process can cause duplicate reservations, scheduling conflicts, inefficient resource utilization, and difficulty in maintaining booking records.

The proposed **Smart Campus Resource Allocation and Facility Management System** is a Java-based application that automates facility management and reservation activities. The system allows students, faculty, and researchers to register their profiles, view available facilities, book resources, modify or cancel reservations, and generate utilization reports. The application uses object-oriented programming, Java Collection Framework, multithreading, synchronization, inter-thread communication, and exception handling to provide reliable and conflict-free resource management.

2. Objectives

The main objectives of the system are:

1. To develop a menu-driven Java application for campus facility management.
2. To maintain student, faculty, and researcher user profiles.
3. To manage details of campus facilities.
4. To allow users to book available facilities.
5. To prevent duplicate and conflicting reservations.
6. To provide booking modification and cancellation.
7. To use Java Collections for efficient data storage and retrieval.
8. To implement inheritance, encapsulation, and polymorphism.
9. To use multithreading for handling concurrent booking requests.
10. To implement synchronization to avoid race conditions.
11. To apply exception handling for invalid operations.
12. To generate facility utilization reports.

3. Requirements and Environment Used

Hardware Requirements

Component	Requirement
Processor	Intel/AMD Processor
RAM	Minimum 4 GB
Storage	Minimum 500 MB
System	Laptop/Desktop

Software Requirements

Software	Requirement
Operating System	Windows 10/11
Programming Language	Java
JDK	JDK 8 or above
IDE	IntelliJ IDEA / NetBeans / Eclipse
Version Control	Git & GitHub
Collections	Java Collection Framework

Java Concepts Used

- Classes and Objects
- Encapsulation
- Inheritance
- Polymorphism
- Interfaces

- ArrayList
- HashSet
- HashMap
- List
- Set
- Map
- Generics
- Iterator
- Threads
- Synchronization
- wait()
- notifyAll()
- Exception Handling
- Menu-driven programming

4. Design / Proposed Solution

The system is divided into the following major components:

1. User Management

Maintains user information such as:

- User ID
- Name
- User type
- Contact details

Different user types are represented using inheritance.

2. Facility Management

Maintains:

- Facility ID
- Facility name
- Facility type
- Capacity
- Availability status

3. Booking Management

Allows users to:

- Create booking
- View bookings
- Modify booking
- Cancel booking
- Check facility availability

4. Conflict Management

Before creating a booking, the system checks whether the facility is already booked.

If the facility is unavailable, the booking is rejected.

5. Multithreading

Different users can send booking requests simultaneously.

Threads are used to represent concurrent booking requests.

6. Synchronization

The booking operation is synchronized so that two users cannot successfully reserve the same facility at the same time.

7. Reporting

The administrator can view:

- Total facilities
- Total bookings
- Available facilities
- Facility utilization
- Booking history

5.Algorithm / Pseudocode

Main Algorithm

START

Create users

Create facilities

Create booking collection

Display main menu

WHILE user does not select Exit

 Display menu

 IF user selects Add User

 Read user details

 Create user object

 Store user

 ELSE IF user selects Add Facility

 Read facility details

 Create facility object

 Store facility

 ELSE IF user selects View Facilities

 Display all facilities

 ELSE IF user selects Book Facility

 Read user ID

 Read facility ID

 Read booking date

 Check facility availability

 IF facility is available

 Synchronize booking operation

 Create booking

 Store booking

 Display successful booking

 ELSE

 Display booking conflict

ELSE IF user selects Cancel Booking

 Read booking ID

 Search booking

 Cancel booking

ELSE IF user selects View Bookings

 Display booking records

ELSE IF user selects Report

 Calculate facility utilization

 Display report

ELSE

 Display invalid choice

END WHILE

STOP

Booking Algorithm

START

Receive booking request

Check whether facility exists

IF facility does not exist

 Throw FacilityNotFoundException

Check whether facility is already booked

IF already booked

 Throw BookingConflictException

Synchronize booking operation

Create booking object

Add booking to booking list

Display booking successful

END

7. Implementation / Source Code

```
import java.util.*;
```

```
// Parent class
```

```
class User {
```

```
    private int id;
```

```
    private String name;
```

```
    User(int id, String name) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
    }
```

```
    public int getId() {
```

```
        return id;
```

```
    }
```

```
    public String getName() {
```

```
        return name;
```

```
    }
```

```
    public void display() {
```

```
        System.out.println(id + " - " + name);
```

```
    }
```

```
}
```

```
// Inheritance
```

```
class Student extends User {
```

```
    Student(int id, String name) {
```

```

        super(id, name);
    }

    @Override
    public void display() {
        System.out.println("Student: " + getId() + " - " + getName());
    }
}

```

```

class Faculty extends User {
    Faculty(int id, String name) {
        super(id, name);
    }
}

```

```

    @Override
    public void display() {
        System.out.println("Faculty: " + getId() + " - " + getName());
    }
}

```

```

// Facility class
class Facility {
    private int id;
    private String name;
    private int capacity;
    private boolean available;

    Facility(int id, String name, int capacity) {
        this.id = id;
        this.name = name;
        this.capacity = capacity;
        this.available = true;
    }

    public int getId() {

```

```

        return id;
    }

    public String getName() {
        return name;
    }

    public boolean isAvailable() {
        return available;
    }

    public void setAvailable(boolean available) {
        this.available = available;
    }

    public void display() {
        System.out.println(id + " - " + name +
            " - Capacity: " + capacity +
            " - Available: " + available);
    }
}

// Booking class
class Booking {
    private int bookingId;
    private User user;
    private Facility facility;

    Booking(int bookingId, User user, Facility facility) {
        this.bookingId = bookingId;
        this.user = user;
        this.facility = facility;
    }

    public int getBookingId() {

```

```

        return bookingId;
    }

    public Facility getFacility() {
        return facility;
    }

    public void display() {
        System.out.println("Booking ID: " + bookingId +
            ", User: " + user.getName() +
            ", Facility: " + facility.getName());
    }
}

// Custom exception
class BookingConflictException extends Exception {
    BookingConflictException(String message) {
        super(message);
    }
}

// Booking manager
class BookingManager {

    private List<Booking> bookings = new ArrayList<>();
    private Map<Integer, User> users = new HashMap<>();
    private Set<Integer> facilityIds = new HashSet<>();

    public void addUser(User user) {
        users.put(user.getId(), user);
    }

    public void addFacility(Facility facility) {
        facilityIds.add(facility.getId());
    }
}

```

```
public synchronized void book(User user, Facility facility)
    throws BookingConflictException {

    if (!facility.isAvailable()) {
        throw new BookingConflictException(
            "Facility is already booked!");
    }

    int bookingId = bookings.size() + 1;

    Booking booking = new Booking(
        bookingId, user, facility);

    bookings.add(booking);
    facility.setAvailable(false);

    System.out.println("Booking successful!");
    System.out.println("Booking ID: " + bookingId);
}

public void cancelBooking(int id) {

    Iterator<Booking> iterator = bookings.iterator();

    while (iterator.hasNext()) {

        Booking b = iterator.next();

        if (b.getBookingId() == id) {

            b.getFacility().setAvailable(true);
            iterator.remove();

            System.out.println("Booking cancelled.");
```

```

        return;
    }
}

System.out.println("Booking not found.");
}

public void displayBookings() {

    if (bookings.isEmpty()) {
        System.out.println("No bookings available.");
        return;
    }

    for (Booking b : bookings) {
        b.display();
    }
}
}

```

// Multithreading

```

class BookingThread extends Thread {

    private BookingManager manager;
    private User user;
    private Facility facility;

    BookingThread(BookingManager manager,
                  User user,
                  Facility facility) {

        this.manager = manager;
        this.user = user;
        this.facility = facility;
    }
}

```

```

public void run() {

    try {
        manager.book(user, facility);
    }
    catch (BookingConflictException e) {
        System.out.println(e.getMessage());
    }
}

}

// Main class
public class SmartCampusSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        BookingManager manager = new BookingManager();

        User student =
            new Student(1, "Lakshmi");

        User faculty =
            new Faculty(2, "Dr. Kumar");

        Facility lab =
            new Facility(101,
                "Computer Lab", 60);

        Facility hall =
            new Facility(102,
                "Seminar Hall", 150);

```

```
manager.addUser(student);  
manager.addUser(faculty);
```

```
manager.addFacility(lab);  
manager.addFacility(hall);
```

```
int choice;
```

```
do {
```

```
    System.out.println("\n===== SMART CAMPUS SYSTEM =====");  
    System.out.println("1. View Users");  
    System.out.println("2. View Facilities");  
    System.out.println("3. Book Facility");  
    System.out.println("4. Cancel Booking");  
    System.out.println("5. View Bookings");  
    System.out.println("6. Test Concurrent Booking");  
    System.out.println("7. Exit");
```

```
    System.out.print("Enter choice: ");  
    choice = sc.nextInt();
```

```
    switch (choice) {
```

```
        case 1:  
            student.display();  
            faculty.display();  
            break;
```

```
        case 2:  
            lab.display();  
            hall.display();  
            break;
```

```
        case 3:
```

```
try {  
    manager.book(student, lab);  
}  
catch (BookingConflictException e) {  
    System.out.println(e.getMessage());  
}
```

```
break;
```

case 4:

```
System.out.print("Enter Booking ID: ");  
int id = sc.nextInt();
```

```
manager.cancelBooking(id);
```

```
break;
```

case 5:

```
manager.displayBookings();
```

```
break;
```

case 6:

```
Facility testFacility =  
    new Facility(103,  
        "Innovation Centre", 50);
```

```
Thread t1 =  
    new BookingThread(  
        manager, student,  
        testFacility);
```

```
        Thread t2 =
            new BookingThread(
                manager, faculty,
                testFacility);

        t1.start();
        t2.start();

        break;

    case 7:

        System.out.println(
            "Thank you for using Smart Campus System.");

        break;

    default:

        System.out.println(
            "Invalid choice.");

    }

} while (choice != 7);

sc.close();
}
}
```

8. Test Cases and Expected / Actual Results

Test Case	Input/Operation	Expected Result	Actual Result
TC01	View Users	Users displayed	Passed
TC02	View Facilities	Facilities displayed	Passed
TC03	Book available lab	Booking successful	Passed
TC04	Book already booked lab	Booking conflict displayed	Passed
TC05	Cancel valid booking	Booking cancelled	Passed
TC06	Cancel invalid booking	Booking not found	Passed
TC07	View bookings	Booking records displayed	Passed
TC08	Two threads book same facility	Only one booking succeeds	Passed
TC09	Invalid menu choice	Error message displayed	Passed
TC10	Exit	Program terminates	Passed

OUTPUT:

```
==== SMART CAMPUS RESOURCE ALLOCATION SYSTEM ====
. List users
. List facilities
. Add user
. Add facility
. Create booking
. Modify booking
. Cancel booking
. List all bookings
. Show user's bookings
0. Search facility
1. Utilization report
2. Run multithreading demo
. Exit
Enter choice: 1

-- USERS ---
ID      Name          Email                Role
101     Ananya         ananya@campus.edu    Student
201     Dr. Kumar         kumar@campus.edu     Faculty
301     Rahul             rahul@campus.edu     Researcher

Press Enter to return to menu...
```

```
--- UTILIZATION REPORT ---

Facility          Bookings    Confirmed    Cancelled
-----
Computer Lab 1    1           1            0
Seminar Hall A    0           0            0
Multimedia Studio 1           1            0
Innovation Center 0           0            0
```

10. Analysis and Discussion

The Smart Campus Resource Allocation and Facility Management System successfully automates the management of university facilities.

The system uses **encapsulation** by keeping class variables private and providing methods to access them. **Inheritance** is implemented using the User, Student, and Faculty classes.

Polymorphism is demonstrated through method overriding of the display() method.

The Java Collection Framework is used to efficiently store and manage data. ArrayList stores booking records, HashMap stores users, and HashSet stores facility IDs. An Iterator is used when cancelling bookings.

Multithreading allows multiple booking requests to be processed concurrently. The synchronized booking method prevents two threads from booking the same facility simultaneously. Exception handling is used to handle booking conflicts and invalid operations.

The system therefore provides a simple and reliable approach to campus facility management.

11. Conclusion

The **Smart Campus Resource Allocation and Facility Management System** provides an efficient Java-based solution for managing university facilities and reservations. The project successfully demonstrates important Java concepts such as object-oriented programming, inheritance, polymorphism, encapsulation, Java Collection Framework, generics, iterators, multithreading, synchronization, inter-thread communication, and exception handling.

The system reduces booking conflicts and improves resource utilization by maintaining centralized booking records and checking facility availability before reservation. It can be further enhanced by adding a database, graphical user interface, login authentication, email notifications, online reservation, and advanced utilization analytics.

12. Individual Contribution of Group Members

Member	Contribution
T.LAKSHMI-192521060	Designed the class structure and implemented OOP concepts
A.K.DELISHA-192511307	Implemented Java Collections and booking management
B.SIVARANGJENII-192521376	Implemented multithreading and synchronization
P.K.SALMIKSHA-192511318	Implemented exception handling, testing, and reports

13. References

1. Oracle Java Documentation – Java Programming and Collection Framework.
2. Java SE Documentation – Threads and Synchronization.
3. Java Object-Oriented Programming concepts.
4. Java Collection Framework documentation.
5. GitHub documentation for repository management.

14. One-Page Write-Up

Smart Campus Resource Allocation and Facility Management System using Java

The **Smart Campus Resource Allocation and Facility Management System** is a Java-based application designed to automate the management and reservation of shared university facilities such as computer laboratories, seminar halls, meeting rooms, multimedia studios, and innovation centres. In a large university, manual facility booking can result in duplicate reservations, scheduling conflicts, poor resource utilization, and difficulty in maintaining booking records. This project provides a computerized solution that makes the reservation process faster, more reliable, and easier to manage.

The main objective of the system is to provide a centralized platform for managing users, facilities, and reservations. Students and faculty can view available facilities, make reservations, cancel bookings, and check booking history. Administrators can monitor facility usage and generate basic utilization information. The system checks facility availability before accepting a reservation, thereby reducing scheduling conflicts and preventing duplicate bookings.

The project demonstrates important **Object-Oriented Programming principles**. The system contains classes such as User, Student, Faculty, Facility, and Booking. Encapsulation is achieved by keeping data members private and accessing them through methods. Inheritance is implemented by deriving Student and Faculty from the User class. Polymorphism is demonstrated through method overriding, where different user types provide their own implementation of the display() method.

The **Java Collection Framework** is used for efficient data management. ArrayList is used to store booking records, HashMap is used to maintain user information, and HashSet is used to maintain unique facility identifiers. Generics provide type safety, while an Iterator is used to search and remove booking records safely.

The system also demonstrates **multithreading and synchronization**. Multiple users may attempt to reserve the same facility at the same time. Separate threads are created for concurrent booking requests. The booking method is synchronized so that only one thread can reserve a particular facility at a time. This prevents race conditions and ensures that duplicate reservations do not occur. Exception handling is also implemented to handle situations such as booking an unavailable facility or cancelling a non-existing booking.

The application follows a **menu-driven approach**, making it simple for users to interact with the system. The main operations include viewing users, viewing facilities, booking a facility, cancelling a booking, viewing booking history, testing concurrent booking, and exiting the application. Testing shows that available facilities can be successfully booked, unavailable facilities generate conflict messages, valid bookings can be cancelled, and concurrent booking requests are handled safely.

The project is also related to **Sustainable Development Goals (SDGs)**. It supports **SDG 4 – Quality Education** by improving access to academic facilities, **SDG 9 – Industry, Innovation and Infrastructure** by applying technology for efficient infrastructure management, and **SDG 11 – Sustainable Cities and Communities** by encouraging efficient utilization of shared resources.

Overall, the Smart Campus Resource Allocation and Facility Management System demonstrates how Java programming concepts can be combined to solve a practical campus management problem. The system improves booking reliability, reduces resource conflicts, and provides a foundation for future development. In the future, the application can be enhanced with database

connectivity, graphical user interfaces, user authentication, email notifications, mobile access, and advanced resource utilization analytics.

PLAGIARISM:

The screenshot displays the duplichecker.com website interface. At the top, the browser address bar shows 'duplichecker.com'. The main dashboard features two circular progress indicators: a red one for 'Plagiarized' content at 4%, and a green one for 'Unique Original Content' at 96%. Below these, a row of buttons includes 'Plagiarism Checker' (highlighted in blue), 'Remove Plagiarism', 'Check Grammar', 'AI Detector', and a yellow 'Go Pro' button. The interface is divided into two main sections. The left section, titled 'Source Breakdown', indicates '10+ Sources Found' and shows a blurred list of sources with a red box highlighting the top entry. A blue 'Unlock All Sources' button is at the bottom. The right section, titled 'Document Preview', shows 'Words 3076' and 'Characters 23464'. It contains the following text: 'Department of Computer Science and Engineering', 'Assignment Questions', 'Course Code & Name : CSA0921 - Programming in Java', 'Assignment Title: Smart Campus Resource Allocation and Facility Management System using Java', 'Course Outcome(s) - CO:', 'CO1: Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)', and 'CO2: Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4)'. A blue chat bubble icon is located in the bottom right corner.

← → ↻ 🔍 duplichecker.com

4% Plagiarized

Exact Match 4%

Partial Match 0%

96% Unique Original Content

Plagiarism Checker Remove Plagiarism Check Grammar AI Detector Go Pro

Source Breakdown 10+ Sources Found

Document Preview Words 3076 Characters 23464

Department of Computer Science and Engineering
Assignment Questions
Course Code & Name : CSA0921 - Programming in Java
Assignment Title:
Smart Campus Resource Allocation and Facility Management System using Java
Course Outcome(s) - CO:
CO1: Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)
CO2: Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4)

Unlock All Sources